

Breast Tumor Classification

Python code handout

Source code: https://github.com/vivek0586/AI_In_Healthcare_Project/blob/main/src/analyze.py

Dependencies: https://github.com/vivek0586/AI_In_Healthcare_Project/blob/main/requirements.txt

Notebook: https://github.com/vivek0586/AI_In_Healthcare_Project/blob/main/notebooks/breast_cancer_tutorial.ipynb

Presentation: https://github.com/vivek0586/AI_In_Healthcare_Project/blob/main/presentation/breast_cancer_ml_tutorial.pptx

Run from the project root: `python src/analyze.py`

Educational benchmark only. This model is not for patient care.

SOURCE CODE | LINE NUMBERS REFER TO `src/analyze.py`

```

1  """Teach a complete, leakage-safe binary classification workflow.
2
3  Run from the project directory with `python src/analyze.py`. The dataset is
4  loaded from scikit-learn, so no private credentials or manual downloads are
5  needed. Comments explain why each step is present, not only what it does.
6  """
7
8  from __future__ import annotations
9
10 import json
11 import os
12 from pathlib import Path
13
14 # Use a file-rendering backend so plots work in headless terminals and hosted
15 # notebooks. Keep Matplotlib's cache inside the project rather than requiring
16 # write access to a user's home directory.
17 os.environ.setdefault("MPLCONFIGDIR", str(Path(__file__).resolve().parents[1] / ".matplotlib"))
18 import matplotlib
19
20 matplotlib.use("Agg")
21 import matplotlib.pyplot as plt
22 import pandas as pd
23 from sklearn.datasets import load_breast_cancer
24 from sklearn.linear_model import LogisticRegression
25 from sklearn.metrics import (
26     ConfusionMatrixDisplay,
27     accuracy_score,
28     confusion_matrix,
29     precision_score,
30     recall_score,
31     roc_auc_score,
32     roc_curve,
33 )
34 from sklearn.model_selection import train_test_split
35 from sklearn.pipeline import Pipeline
36 from sklearn.preprocessing import StandardScaler
37
38
39 # Make the random split reproducible. Reproducible does not mean universally
40 # representative: it only means this exact split can be regenerated.
41 RANDOM_SEED = 42
42 TEST_SIZE = 0.20
43 DECISION_THRESHOLD = 0.50
44
45
46 def main() -> None:
47     """Load data, fit the model, evaluate it, and save teaching artifacts."""
48     # Anchor paths to this file, not the shell's current directory. This lets
49     # a student run the script from the project root or another directory.
50     project_dir = Path(__file__).resolve().parents[1]
51     output_dir = project_dir / "outputs"
52     output_dir.mkdir(parents=True, exist_ok=True)
53
54     # `as_frame=True` gives us named columns, which makes the later coefficient
55     # table much easier to read than a matrix of anonymous column numbers.
56     dataset = load_breast_cancer(as_frame=True)
57     X = dataset.data

```

```

58
59 # In the scikit-learn copy, target 0 means malignant and target 1 means
60 # benign. We recode explicitly so that "positive" consistently means
61 # malignant. This decision controls what sensitivity and recall mean.
62 malignant_code = list(dataset.target_names).index("malignant")
63 y = (dataset.target == malignant_code).astype(int)
64
65 # Keep a record of the starting data shape and labels. These checks catch
66 # common mistakes such as accidentally swapping the positive class.
67 assert X.shape == (569, 30), f"Unexpected data shape: {X.shape}"
68 assert int(y.sum()) == 212, "Positive label should represent 212 malignant cases."
69 assert not X.isna().any().any(), "This tutorial assumes no missing feature values."
70
71 # Split before learning any preprocessing parameters. Stratification keeps
72 # roughly the same benign/malignant proportions in both partitions.
73 X_train, X_test, y_train, y_test = train_test_split(
74     X,
75     y,
76     test_size=TEST_SIZE,
77     random_state=RANDOM_SEED,
78     stratify=y,
79 )
80
81 # StandardScaler learns means and standard deviations. Placing it inside
82 # the Pipeline ensures it learns from training rows only; scaling the full
83 # dataset before the split would leak information from the test set.
84 # Logistic regression is a useful teaching model: it is fast, and its
85 # standardized coefficients can be inspected (with careful caveats).
86 model = Pipeline(
87     steps=[
88         ("scale", StandardScaler()),
89         (
90             "classifier",
91             LogisticRegression(
92                 max_iter=5_000,
93                 class_weight=None,
94                 random_state=RANDOM_SEED,
95             ),
96         ),
97     ]
98 )
99
100 # Fit once on training rows. The held-out test rows remain untouched until
101 # the final evaluation below.
102 model.fit(X_train, y_train)
103
104 # Score each test sample with a probability, then apply a clearly stated
105 # threshold to turn that probability into a binary prediction.
106 malignant_probability = model.predict_proba(X_test)[:, 1]
107 y_pred = (malignant_probability >= DECISION_THRESHOLD).astype(int)
108
109 # In this tutorial malignant is positive: recall is sensitivity, and
110 # specificity measures the fraction of benign samples predicted benign.
111 tn, fp, fn, tp = confusion_matrix(y_test, y_pred, labels=[0, 1]).ravel()
112 sensitivity = recall_score(y_test, y_pred, pos_label=1)
113 specificity = tn / (tn + fp)
114 metrics = {
115     "dataset": "Wisconsin Diagnostic Breast Cancer (scikit-learn copy)",
116     "positive_class": "malignant",
117     "samples_total": int(len(y)),
118     "features": int(X.shape[1]),
119     "train_samples": int(len(y_train)),
120     "test_samples": int(len(y_test)),
121     "test_malignant": int(y_test.sum()),
122     "test_benign": int((y_test == 0).sum()),
123     "random_seed": RANDOM_SEED,
124     "test_size": TEST_SIZE,
125     "decision_threshold": DECISION_THRESHOLD,
126     "accuracy": float(accuracy_score(y_test, y_pred)),
127     "sensitivity_recall": float(sensitivity),
128     "specificity": float(specificity),
129     "precision_positive_predictive_value": float(
130         precision_score(y_test, y_pred, pos_label=1, zero_division=0)
131     ),
132     "roc_auc": float(roc_auc_score(y_test, malignant_probability)),
133     "confusion_matrix_rows_true_0_benign_1_malignant_cols_predicted_0_1": [
134         int(tn), int(fp)],

```

```

135         [int(fn), int(tp)],
136     ],
137 }
138
139 # Save the scores as well as the thresholded predictions. Scores let a
140 # learner explore other thresholds without fitting the model again.
141 predictions = X_test.copy()
142 predictions.insert(0, "true_malignant", y_test)
143 predictions["malignant_probability"] = malignant_probability
144 predictions["predicted_malignant_at_0_50"] = y_pred
145 predictions.to_csv(output_dir / "predictions.csv", index_label="sample_index")
146
147 # Logistic-regression coefficients are expressed on standardized feature
148 # units because the scaler is part of the fitted pipeline. They are useful
149 # for teaching model interpretation, but are not causal effects.
150 coefficient_values = model.named_steps["classifier"].coef_[0]
151 coefficients = pd.DataFrame(
152     {"feature": X.columns, "standardized_coefficient": coefficient_values}
153 ).assign(absolute_coefficient=lambda frame: frame.standardized_coefficient.abs())
154 coefficients.sort_values(
155     "absolute_coefficient", ascending=False
156 ).to_csv(output_dir / "coefficients.csv", index=False)
157
158 # The presentation shows the five largest absolute standardized weights.
159 # Generate the same view from the fitted model so its plotted values and
160 # ordering can always be rebuilt from this script. Bars retain their sign:
161 # positive values push the fitted log-odds toward malignant, conditional
162 # on the other features in the model.
163 top_coefficients = coefficients.nlargest(5, "absolute_coefficient").sort_values(
164     "standardized_coefficient"
165 )
166 coefficient_colors = [
167     "#B85C5C" if value < 0 else "#245B78"
168     for value in top_coefficients["standardized_coefficient"]
169 ]
170 fig, ax = plt.subplots(figsize=(8, 4.8))
171 ax.barh(
172     top_coefficients["feature"],
173     top_coefficients["standardized_coefficient"],
174     color=coefficient_colors,
175 )
176 ax.axvline(0, color="#555555", linewidth=1)
177 ax.set(
178     xlabel="Standardized logistic-regression coefficient",
179     title="Five largest absolute coefficients",
180 )
181 ax.grid(axis="x", alpha=0.2)
182 fig.tight_layout()
183 fig.savefig(output_dir / "coefficients_top5.png", dpi=180)
184 plt.close(fig)
185
186 # The presentation's threshold exercise reuses these held-out probabilities
187 # without refitting. Save the reference counts and metrics so learners can
188 # check their own calculations after attempting the exercise.
189 threshold_rows = []
190 for threshold in (0.25, DECISION_THRESHOLD, 0.75):
191     predicted_at_threshold = (malignant_probability >= threshold).astype(int)
192     threshold_tn, threshold_fp, threshold_fn, threshold_tp = confusion_matrix(
193         y_test, predicted_at_threshold, labels=[0, 1]
194     ).ravel()
195     threshold_rows.append(
196         {
197             "threshold": threshold,
198             "true_negatives": int(threshold_tn),
199             "false_positives": int(threshold_fp),
200             "false_negatives": int(threshold_fn),
201             "true_positives": int(threshold_tp),
202             "sensitivity_recall": float(
203                 threshold_tp / (threshold_tp + threshold_fn)
204             ),
205             "specificity": float(
206                 threshold_tn / (threshold_tn + threshold_fp)
207             ),
208             "accuracy": float(
209                 (threshold_tn + threshold_tp) / len(y_test)
210             ),
211         }

```

```

212     )
213     pd.DataFrame(threshold_rows).to_csv(
214         output_dir / "threshold_comparison.csv", index=False
215     )
216
217     (output_dir / "metrics.json").write_text(
218         json.dumps(metrics, indent=2) + "\n", encoding="utf-8"
219     )
220
221     # Produce a labeled confusion matrix. Rows are truth and columns are model
222     # predictions, so each error type is visible rather than hidden in accuracy.
223     matrix_display = ConfusionMatrixDisplay.from_predictions(
224         y_test,
225         y_pred,
226         labels=[0, 1],
227         display_labels=["Benign", "Malignant"],
228         cmap="Blues",
229         colorbar=False,
230     )
231     matrix_display.ax_.set_title("Held-out test set: confusion matrix")
232     matrix_display.figure_.tight_layout()
233     matrix_display.figure_.savefig(output_dir / "confusion_matrix.png", dpi=180)
234     plt.close(matrix_display.figure_)
235
236     # ROC curves show the trade-off between sensitivity and false-positive
237     # rate across thresholds. AUC summarizes ranking performance, not clinical
238     # usefulness, calibration, or an appropriate operating threshold.
239     false_positive_rate, true_positive_rate, _ = roc_curve(
240         y_test, malignant_probability, pos_label=1
241     )
242     fig, ax = plt.subplots(figsize=(6.5, 5))
243     ax.plot(
244         false_positive_rate,
245         true_positive_rate,
246         color="#245B78",
247         linewidth=2.5,
248         label=f"Logistic regression (AUC = {metrics['roc_auc']:.3f})",
249     )
250     ax.plot([0, 1], [0, 1], linestyle="--", color="#777777", label="Chance ranking")
251     ax.set(
252         xlabel="False-positive rate (1 - specificity)",
253         ylabel="Sensitivity (recall)",
254         title="ROC curve on the held-out test set",
255         xlim=(0, 1),
256         ylim=(0, 1.02),
257     )
258     ax.legend(loc="lower right")
259     ax.grid(alpha=0.2)
260     fig.tight_layout()
261     fig.savefig(output_dir / "roc_curve.png", dpi=180)
262     plt.close(fig)
263
264     # Print a concise summary so a learner can check the run without opening
265     # the JSON file. More complete artifacts remain available in outputs/.
266     print(json.dumps(metrics, indent=2))
267
268
269 if __name__ == "__main__":
270     main()

```